

The Journey of Building Qubit Simulator

Brian Hernandez

Latest Update: June 17, 2026

1 Introduction

This document chronicles my journey in creating **Qubit Simulator**. It captures my thought process, all my fails, successes, and ideas for the future to come.

2 The Language

Since finding the probability amplitudes of a qubit without explicit calculation is impossible, it is necessary to use matrix multiplication to find the state after each operation applied. While I could use Python or a library that already implements matrix multiplication, what's the fun in that?

So the first step is to make a library that implements matrix multiplication, and what better language to do it in than C. It's fast, simple, and straightforward. It has all I need: no more, no less... Or so I thought. I realized shortly after writing my first few classes that using Webassembly would be too tedious and using http server communication would over-complicate this app for no reason. So I did what I least wanted to do: I scrapped using C and resorted to writing everything in Typescript instead.

3 The Clases

So with the language choice out of the way, we can start implementing some linear algebra: The first step is to use a class to implement complex numbers. It simply has to have a real part and imaginary part:

3.1 Converting Vector to Spherical Coordinates

We can represent any qubit on the Bloch Sphere using the spherical coordinates (θ, ϕ, r) . Since $|\alpha|^2 + |\beta|^2 = 1$, r is always 1 and so for any pure state qubit, the resulting coordinate will always be on the surface of the Bloch Sphere.

Any qubit can be represented in the following form: $\cos(\frac{\theta}{2})|0\rangle + e^{i\phi}\sin(\frac{\theta}{2})|1\rangle$, where ϕ is the relative phase between $|0\rangle$ and $|1\rangle$. θ will be limited to values between 0 and π , so the fact that $\cos$ is a symmetric function does not matter: the negative value of θ can always be rejected.

Now that we know the value of θ , we can plug it into $e^{i\phi}\sin(\frac{\theta}{2}) = \beta$ to solve for the only unknown variable: ϕ .

Solving this gives us: $\phi = -i \cdot \ln(\frac{\beta}{\sin(\frac{\theta}{2})})$, where we use $-i$ as the inverse of i , and use $\phi = 0$ if $\theta = 0$.

Now going back to the part about ϕ being the relative phase, we can easily make sure this assumption always holds true by factoring out any complex number